

API RESTful de gestión de libros con Spring Boot 2, MongoDB, Spring Security, HATEOAS y OpenAPI.

<https://github.com/edwise/complete-spring-project>

Actividades de la Unidad 2: AOP sobre `complete-spring-project`

Contexto del proyecto

El proyecto expone un servicio REST `/api/books/` con:

- **Controller** → `BookController`
- **Service** → `BookService`
- **Repository** → MongoDB con Spring Data
- **Security** → autenticación básica
- **Logs** → SLF4J/Logback

Esto da pie natural a los **4 casos de uso clásicos de AOP**:

Paso 0 — Agregar la dependencia (ya incluida en Spring Boot)

Spring Boot incluye AspectJ por defecto a través de `spring-boot-starter-aop`. Solo hay que agregarlo al `pom.xml`:

```
xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

Y habilitar AOP con la anotación en la clase principal (aunque Spring Boot lo hace automáticamente con el starter):

```
java
@EnableAspectJAutoProxy
@SpringBootApplication
public class Application { ... }
```

Ejercicios graduados por dificultad

Ejercicio 1 — Logging de métodos del servicio (*Básico*)

Concepto: `@Before`, `@After`, `@Around`, Pointcut expressions

Tarea: Crear un aspecto que registre en el log cada vez que se llame a cualquier método de `BookService`, mostrando el nombre del método y sus argumentos.

```
java
@Aspect
@Component
public class LoggingAspect {

    private static final Logger log = LoggerFactory.getLogger(LoggingAspect.class);

    @Before("execution(* com.edwise.completespring.service..*(..))")
    public void logBeforeServiceCall(JoinPoint jp) {
        log.info("Llamando a: {} con args: {}",
            jp.getSignature().getName(),
            Arrays.toString(jp.getArgs()));
    }

    @AfterReturning(pointcut = "execution(* com.edwise.completespring.service..*(..))",
        returning = "result")
    public void logAfterServiceCall(JoinPoint jp, Object result) {
        log.info("Método {} retornó: {}", jp.getSignature().getName(), result);
    }
}
```

Punto de discusión: ¿Por qué es mejor hacerlo aquí que poner `log.info()` en cada método del servicio?

Ejercicio 2 — Medición de tiempos de ejecución (*Intermedio*)

Concepto: `@Around`, `ProceedingJoinPoint`

Tarea: Crear un aspecto que mida cuánto tiempo tarda cada operación del controller y lo registre. Si tarda más de un umbral (ej. 200ms), emitir una advertencia.

```
java
@Aspect
@Component
public class PerformanceAspect {

    private static final Logger log = LoggerFactory.getLogger(PerformanceAspect.class);
    private static final long THRESHOLD_MS = 200;

    @Around("execution(* com.edwise.completespring.controllers..*(..))")
    public Object measureTime(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = pjp.proceed();
        long elapsed = System.currentTimeMillis() - start;

        if (elapsed > THRESHOLD_MS) {
            log.warn("Método lento: {} tardó {}ms", pjp.getSignature(), elapsed);
        } else {
            log.info("{} tardó {}ms", pjp.getSignature().getName(), elapsed);
        }
        return result;
    }
}
```

```
}  
}
```

Ejercicio 3 — Auditoría con anotación personalizada (*Intermedio-Avanzado*)

Concepto: Anotaciones custom + @Around

Tarea: Crear una anotación @Auditable y aplicarla a los métodos que modifican datos (crear, actualizar, eliminar libro). El aspecto debe registrar quién realizó la operación usando el contexto de Spring Security.

```
java  
// 1. Definir la anotación  
@Target(ElementType.METHOD)  
@Retention(RetentionPolicy.RUNTIME)  
public @interface Auditable {  
    String action() default "";  
}  
  
// 2. Aplicarla en el servicio  
@Auditable(action = "CREATE_BOOK")  
public Book createBook(Book book) { ... }  
  
@Auditable(action = "DELETE_BOOK")  
public void deleteBook(Long id) { ... }  
  
// 3. El aspecto  
@Aspect  
@Component  
public class AuditAspect {  
  
    @Around("@annotation(auditable)")  
    public Object audit(ProceedingJoinPoint pjp, Auditable auditable) throws Throwable {  
        String user = SecurityContextHolder.getContext()  
            .getAuthentication().getName();  
  
        log.info("AUDITORÍA | usuario: {} | acción: {} | args: {}",  
            user, auditable.action(), Arrays.toString(pjp.getArgs()));  
  
        Object result = pjp.proceed();  
  
        log.info("AUDITORÍA | acción: {} | completada exitosamente", auditable.action());  
        return result;  
    }  
}
```

Ejercicio 4 — Manejo centralizado de excepciones con AOP (*Avanzado*)

Concepto: @AfterThrowing

Tarea: Crear un aspecto que intercepte excepciones lanzadas en cualquier capa y las registre con información detallada (clase, método, excepción) antes de que el @ControllerAdvice las maneje.

```

java
@Aspect
@Component
public class ExceptionLoggingAspect {

    private static final Logger log = LoggerFactory.getLogger(ExceptionLoggingAspect.class);

    @AfterThrowing(
        pointcut = "execution(* com.edwise.completespring..*(..))",
        throwing = "ex"
    )
    public void logException(JoinPoint jp, Exception ex) {
        log.error("Excepción en {}.{} → {}: {}",
            jp.getTarget().getClass().getSimpleName(),
            jp.getSignature().getName(),
            ex.getClass().getSimpleName(),
            ex.getMessage());
    }
}

```

Punto de discusión: ¿Cómo se diferencia esto del `@ControllerAdvice` que ya existe en el proyecto?

Ejercicio 5 — Caché manual con AOP (*Avanzado*)

Concepto: `@Around` como interceptor de infraestructura

Tarea: Implementar un caché en memoria simple para `getBook(Long id)`, evitando llamadas repetidas a MongoDB.

```

java
@Aspect
@Component
public class CacheAspect {

    private final Map<Object, Object> cache = new ConcurrentHashMap<>();

    @Around("execution(* com.edwise.completespring.service.*getBook(..))")
    public Object cache(ProceedingJoinPoint pjp) throws Throwable {
        Object key = Arrays.toString(pjp.getArgs());

        if (cache.containsKey(key)) {
            log.info("☐ Cache HIT para key: {}", key);
            return cache.get(key);
        }

        Object result = pjp.proceed();
        cache.put(key, result);
        log.info("☐ Cache MISS → almacenado key: {}", key);
        return result;
    }
}

```

□ Resumen de conceptos cubiertos

Ejercicio	Advice	Concepto clave
1 - Logging	@Before / @AfterReturning	Pointcut expressions, JoinPoint
2 - Tiempos	@Around	ProceedingJoinPoint, proceed()
3 - Auditoría	@Around + anotación	Custom annotations, SecurityContext
4 - Excepciones	@AfterThrowing	Manejo de errores transversal
5 - Caché	@Around	AOP como infraestructura